

# **Introduction to C++**

**Mao**

# Outline



- **History of C++**
- **Coding Environment**
- **A Simple C++ Program**
- **C++ Program**
  - **Keywords**
  - **Special Characters**
  - **Examples**
- **Reading Input from the Keyboard**
- **C++ Program—Special Characters**

# Outline



- **Named Constants**
- **Numeric Data Types**
- **Numeric Type Conversions**
- **Assignment Statements and Assignment Expressions**
- **Increment and Decrement Operators**
- **Development Cycle**

# History of C++



**C++ is a general-purpose, object-oriented programming language.**

**C, C++, Java, and C# are related. C++ evolved from C. Java was modeled after C++. C# is a subset of C++, with some features similar to Java. If you know one of these languages, it is easy to learn the others.**

# History of C++



**C++ is an extension of C, developed by Bjarne Stroustrup at Bell Laboratories during 1983–1985. C++ added a number of features that improved the C language. Most important, it added the support of using classes for object-oriented programming.**

**A new standard, known as C++11, was approved by ISO in 2011. C++11 added new features into the core language and standard library. These new features are very useful for advanced C++ programming. Since 2012, C++ has been on a three-year release schedule. There are C++14, C++17, C++20 and so forth.**

# Coding Environment

**Integrated development environment (IDE) for editing, compiling, running, and debugging programs: Visual Studio, Xcode, Eclipse, Code::Blocks, ...**

**For Windows, Cygwin (C++ compiler) may need to be installed with an IDE like Microsoft Visual Studio Code.**

**For MacOS, Xcode bundled with GCC can be installed and used directly.**

**Linux comes with GCC.**

**Eclipse (for C/C++ developers) works on all operating systems. If a different version of Eclipse has been installed, just install the C/C++ add-ons through Eclipse's add-on manager.**

# A Simple C++ Program

Let us begin with a simple C++ program that displays the message **Welcome to C++!** on the **console**.

The word console is **an old computer term**. It refers to a **computer's text entry and display device**.

- Console **input** means to receive input **from the keyboard**
- and console **output** means to display **output to the monitor**.

# A Simple C++ Program

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      // Display Hello World! to the console
7      cout << "Hello World!" << endl;
8
9      return 0;
10 }
```

# A Simple C++ Program

```
1  #include <iostream>
```

This line is a **compiler preprocessor directive** that tells the **compiler** to include the **iostream** library in this program, which is **needed to support console input and output** (so that we don't need to implement them by ourselves.)

C++ library contains **predefined code** for developing C++ programs. The library like **iostream** is called **a header file** in C++, because it is usually included **at the head of a program**.

# A Simple C++ Program

2    `using namespace std;`

This line tells the compiler to **use the standard namespace**. `std` is an abbreviation of standard. **Namespace is a mechanism to avoid naming conflicts in a large program**. The names `cout` and `endl` in line 7 are defined in the `iostream` library in the standard namespace.

If line 2 is missing, we need to use to switch from `cout` to `std::cout` and from `endl` to `std::endl` in the program.

# A Simple C++ Program

```
4  int main()
5  {
6      // Display Hello World! to the console
7      cout << "Hello World!" << endl;
8
9      return 0;
10 }
```

**Every** C++ program is **executed from a main function**. The main function defined in lines 4–10 contains two statements. They are enclosed in a block that **starts with a left brace, {**, (line 5) and **ends with a right brace, }** (line 10).

# A Simple C++ Program

```
4  int main()
5  {
6      // Display Welcome to C++! to the console
7      cout << "Welcome to C++!" << endl;
8
9      return 0;
10 }
```

Every statement in C++ **must end with a semicolon (;)**, known as the **statement terminator**.

**cout (pronounced see-out)** stands for **console output**. The **<< operator**, referred to as the **stream insertion operator**, **sends a string to the console**.

# A Simple C++ Program

```
4  int main()
5  {
6      // Display Welcome to C++! to the console
7      cout << "Welcome to C++!" << endl;
8
9      return 0;
10 }
```

**A string must be enclosed in quotation marks.**

**endl stands for end line. Sending endl to the console ends a line and flushes the output buffer to ensure that the output is displayed immediately.**

# A Simple C++ Program

```
9         return 0;
```

This line is placed **at the end of every main function** to exit the program. The **value 0** indicates that the program has **terminated with a successful exit**.

Some compilers will work fine if this statement is omitted; however, other compilers will not. It is a good practice always to include this statement.

# Reading Input from the Keyboard

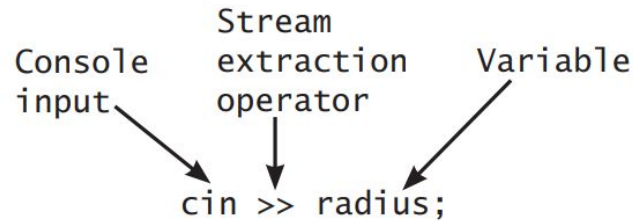
Use the cin object to read a value from the keyboard.

```
1  double radius;  
2  cout << "Enter a radius: ";  
3  cin >> radius;
```

Line 1 defines a double type variable (from  $-1.797693 \times 10^{308}$  to  $1.797693 \times 10^{308}$ ).

Line 3 uses the **cin object** to **read a value from the keyboard**. The **>> symbol**, referred to as the stream extraction operator, **assigns an input to a variable**.

# Reading Input from the Keyboard

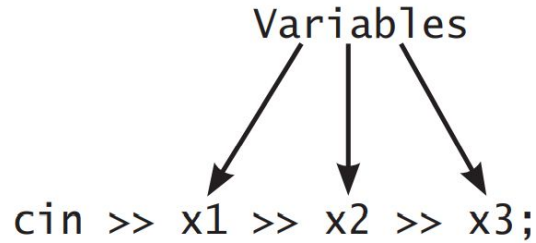


These three lines displays the prompting **message "Enter a radius: "**; the **user** then **enters a number** (e.g. 2), which is **assigned to variable radius**.

The `cin` object causes a program to **wait until data is entered at the keyboard** and **the Enter key is pressed**.

**ComputeAreaWithConsoleInput.cpp**

# Reading Input from the Keyboard



**A single statement can be used to read multiple input. For example, the above statement reads three values into variable `x1`, `x2`, and `x3`:**

**ComputeAverage.cpp**

# C++ Program–Special Characters

| Character | Name                                | Description                                                 |
|-----------|-------------------------------------|-------------------------------------------------------------|
| #         | Pound sign                          | Used in <b>#include</b> to denote a preprocessor directive. |
| <>        | Opening and closing angle brackets  | Encloses a library name when used with <b>#include</b> .    |
| ()        | Opening and closing parentheses     | Used with functions such as <b>main()</b> .                 |
| { }       | Opening and closing braces          | Denotes a block to enclose statements.                      |
| //        | Double slashes                      | Precedes a comment line.                                    |
| <<        | Stream insertion operator           | Outputs to the console.                                     |
| " "       | Opening and closing quotation marks | Wraps a string (i.e., sequence of characters).              |
| ;         | Semicolon                           | Marks the end of a statement.                               |

# Named Constants

The value of a variable may change during the execution of a program, but a named constant, or simply **constant**, represents permanent data that **never changes**.

To declare a constant:

```
const datatype CONSTANTNAME = value;
```

A constant must be **declared and initialized in the same statement**.

# Numeric Data Types

C++ has nine numeric types for integers and floating-point numbers with operators `+`, `-`, `*`, `/`, and `%`.

C++ uses three types for integers: `short`, `int`, and `long`. Each integer type comes in two flavors: `signed` and `unsigned`.

C++ uses three types for floating-point numbers: `float`, `double`, and `long double`. The `double` type is usually `twice as big as float`. So, the double is known as `double precision`, while `float` is `single precision`. For most applications, using the double type is desirable.

## Numeric Data Types

| Name           | Synonymy           | Range                                                                                                                                                              | Storage Size    |
|----------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| short          | short int          | $-2^{15}$ to $2^{15}-1$ ( $-32,768$ to $32,767$ )                                                                                                                  | 16-bit signed   |
| unsigned short | unsigned short int | 0 to $2^{16}-1$ ( $65535$ )                                                                                                                                        | 16-bit unsigned |
| int signed     |                    | $-2^{31}$ to $2^{31}-1$ ( $-2147483648$ to $2147483647$ )                                                                                                          | 32-bit          |
| unsigned       | unsigned int       | 0 to $2^{32}-1$ ( $4294967295$ )                                                                                                                                   | 32-bit unsigned |
| long           | long int           | $-2^{31}$ ( $-2147483648$ ) to $2^{31}-1$ ( $2147483647$ )                                                                                                         | 32-bit signed   |
| unsigned long  | unsigned long int  | 0 to $2^{32}-1$ ( $4294967295$ )                                                                                                                                   | 32-bit unsigned |
| float          |                    | Negative range:<br>$-3.4028235\text{E}+38$ to $-1.4\text{E}-45$<br>Positive range:<br>$1.4\text{E}-45$ to $3.4028235\text{E}+38$                                   | 32-bit IEEE 754 |
| double         |                    | Negative range:<br>$-1.7976931348623157\text{E}+308$ to $-4.9\text{E}-324$<br>Positive range:<br>$4.9\text{E}-324$ to $1.7976931348623157\text{E}+308$             | 64-bit IEEE 754 |
| long double    |                    | Negative range:<br>$-1.18\text{E}+4932$ to $-3.37\text{E}-4932$<br>Positive range:<br>$3.37\text{E}-4932$ to $1.18\text{E}+4932$<br>Significant decimal digits: 19 | 80-bit          |

# Numeric Type Conversions

C++ also allows to convert a value from one type to another explicitly by using a casting operator. The syntax is

```
static_cast<type>(value)
```

```
(type)value      ← C-style cast
```

where **value** is a variable, a literal, or an expression and **type** is the type the value will be converted to. For example,

```
int x;
```

```
x = static_cast<int>(1.7);
```

```
x = (int) 1.7;
```

# Assignment Statements and Assignment Expressions

An assignment statement is also known as an **assignment expression**.

```
x = 1;  
cout << x;
```

==

```
cout << (x = 1);
```

For a value that needs to be assigned to multiple variables:

```
i = j = k = 1;
```

==

```
k = 1;  
j = k;  
i = j;
```

# Increment and Decrement Operators

| Operator     | Name          | Description                                                                               | Example (assume i = 1)                        |
|--------------|---------------|-------------------------------------------------------------------------------------------|-----------------------------------------------|
| <b>++var</b> | preincrement  | Increment <b>var</b> by <b>1</b> and use the new <b>var</b> value in the statement        | <pre>int j = ++i;<br/>// j is 2, i is 2</pre> |
| <b>var++</b> | postincrement | Increment <b>var</b> by <b>1</b> , but use the original <b>var</b> value in the statement | <pre>int j = i++;<br/>// j is 1, i is 2</pre> |
| <b>--var</b> | predecrement  | Decrement <b>var</b> by <b>1</b> and use the new <b>var</b> value in the statement        | <pre>int j = --i;<br/>// j is 0, i is 0</pre> |
| <b>var--</b> | postdecrement | Decrement <b>var</b> by <b>1</b> and use the original <b>var</b> value in the statement   | <pre>int j = i--;<br/>// j is 1, i is 0</pre> |

Consider the following code:

```
int i = 10;  
int newNum = 10 * i++;
```

==

```
int i = 10;  
int newNum = 10 * i;  
i = i + 1;
```

# Increment and Decrement Operators

| Operator     | Name          | Description                                                                               | Example (assume i = 1)                        |
|--------------|---------------|-------------------------------------------------------------------------------------------|-----------------------------------------------|
| <b>++var</b> | preincrement  | Increment <b>var</b> by <b>1</b> and use the new <b>var</b> value in the statement        | <pre>int j = ++i;<br/>// j is 2, i is 2</pre> |
| <b>var++</b> | postincrement | Increment <b>var</b> by <b>1</b> , but use the original <b>var</b> value in the statement | <pre>int j = i++;<br/>// j is 1, i is 2</pre> |
| <b>--var</b> | predecrement  | Decrement <b>var</b> by <b>1</b> and use the new <b>var</b> value in the statement        | <pre>int j = --i;<br/>// j is 0, i is 0</pre> |
| <b>var--</b> | postdecrement | Decrement <b>var</b> by <b>1</b> and use the original <b>var</b> value in the statement   | <pre>int j = i--;<br/>// j is 1, i is 0</pre> |

Consider the following code:

```
int i = 10;  
int newNum = 10 * (++i);
```

==

```
int i = 10;  
i = i + 1;  
int newNum = 10 * i;
```

# Development Cycle



A program need to compiled before it can be executed.

The C++ compiler command performs three tasks in sequence: preprocessing, compiling, and linking.

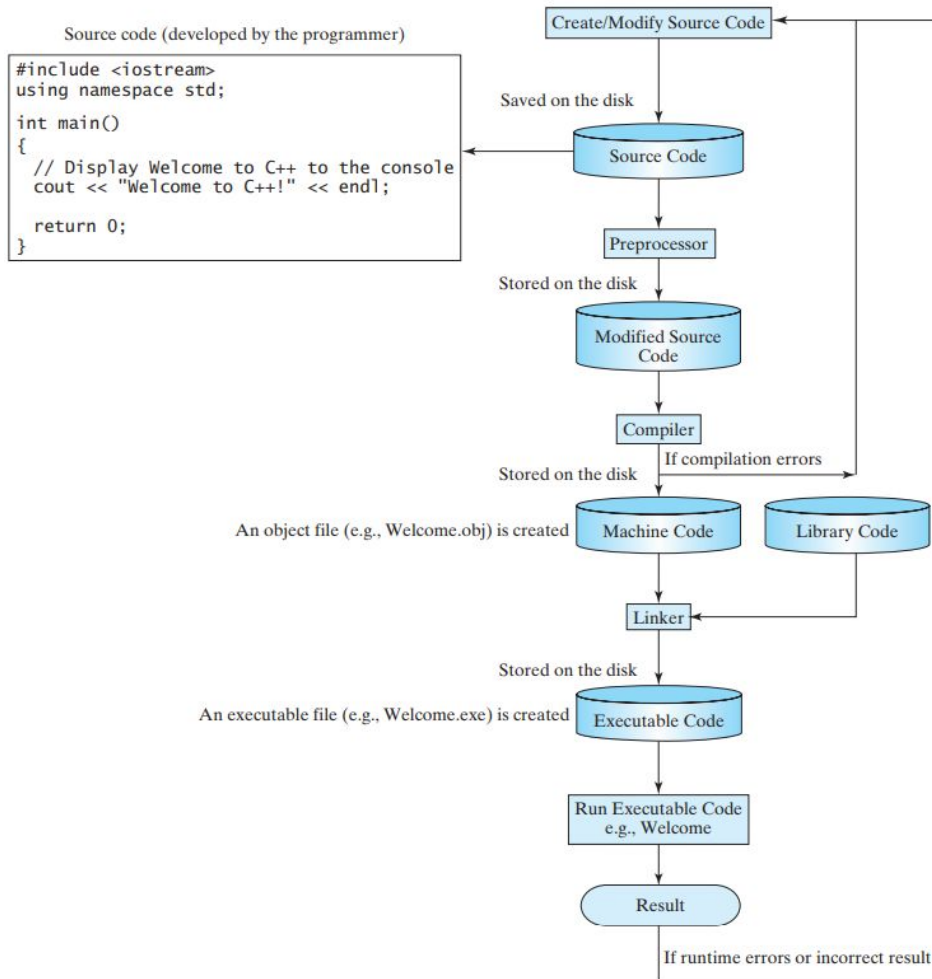
- A **preprocessor** is a program that **processes a source file before it is passed down to the compiler**. For example, **#include** is a directive to tell the compiler to include a library. The preprocessor produces an intermediate file.

# Development Cycle



- The **compiler** then translates the intermediate file into a machine-code file.
- The linker links the machine-code file **with supporting library files to form an executable file.**
  - On Windows, the machine-code file is stored on disk with an **.obj extension**, and **the executable files are stored with an .exe extension.**
  - On UNIX, the machine-code file has an **.o extension** and the executable files **do not have file extensions.**

# Development Cycle



The C++ program-development process consists of creating/modifying source code, compiling, linking and executing programs.

# Reference



- **Revel for Introduction to C++ Programming and Data Structures by Daniel Liang, 5th edition, Pearson, 2022**